Interactive Database Features Walkthrough & Demo Guide

Project: Campus Club Resource Exchange and Intelligent Allocation System | **Team No:** 15 (PES2UG24CS191 & PES2UG24CS215)
Databases: MySQL (Relational & Transactional) + MongoDB (Flexible Resource Specs & Maintenance Logs)

Feature 1 • Views + FULLTEXT Index + Hybrid Query

1. Full-Text Search & Condition Filtering (Catalog Page)

1. Open `http://localhost:3000` (**Resource Catalog**).
2. Point out how each resource card combines two database paradigms:
   - **MySQL Relational Data** (via view `vw_resource_availability`): Resource ID, Name, Owner Club, Room Location, Condition, and live `Available / Total` stock count.
   - **MongoDB Document Data** (via collection `resource_specs`): Schema-less technical specification badges (e.g., `sensor: 24.1MP APS-C CMOS`, `brightness_lumens: 3300`, `polar_pattern: Cardioid`) and usage guidelines.
3. **Test Full-Text Search:** Type `mirrorless gimbal` or `3300-lumen` in the search bar and click **Search Inventory**. Explain that MySQL runs `MATCH(r.name, r.description, r.location) AGAINST(? IN NATURAL LANGUAGE MODE)` using the `FULLTEXT` index `idx_resource_fts`.
4. **Test Condition & Availability Filters:** Select `excellent` from the **All Conditions** dropdown (uses B-Tree index `idx_resource_condition`) and toggle **Available Only** (filters `quantity_currently_available > 0`).

Feature 2 • Hybrid INSERT (MySQL + MongoDB)

2. Registering a New Club Resource Across Both Databases

1. On `http://localhost:3000`, click **"+ Register New Resource"** in the top-right header.
2. Enter the following sample form values:
   - **Owner Club:** `4 — Music Club` | **Category:** `5 — Lighting & Audio`
   - **Resource Name:** `Focusrite Scarlett 2i2 Audio Interface`
   - **Total Quantity:** `2` | **Storage Location:** `Music Studio C1` | **Condition:** `excellent`
   - **Description (MySQL):** `USB-C studio audio interface for podcast and band recording`
   - **Usage Guidelines (MongoDB):** `Keep 48V phantom power off before plugging in condenser mics.`
   - **Flexible Technical Specifications JSON (MongoDB):**

```
{"preamps": 2, "sample_rate": "192kHz/24-bit", "connectivity": "USB-C"}
```

3. Click **Save Resource**. Explain that structured fields are inserted into MySQL's `RESOURCE` table while the custom JSON object is validated and stored in MongoDB's `resource_specs` collection linked by `resource_id`.

Feature 3 • Database-Level CHECK Constraints

3. Demonstrating Database-Level Input Validation

1. Open `http://localhost:3000/requests` (**Reserve & Check Conflicts**).
2. **Test `chk_request_qty CHECK (quantity_requested > 0)`:**
   - Enter `0` or `-1` in **Quantity Needed** and click **2. Submit Request**.
   - Show the error message returned directly by MySQL: `Check constraint 'chk_request_qty' is violated.`
3. **Test `chk_request_time_window CHECK (end_time > start_time)`:**
   - Set **Quantity Needed** back to `1`, set **End Window** earlier than **Start Window**, and click **2. Submit Request**.
   - Show that MySQL rejects invalid time windows: `Check constraint 'chk_request_time_window' is violated.`

Feature 4 • Stored Procedures

4. Scheduling Conflict Detection & Intelligent Alternative Suggestions

1. On `http://localhost:3000/requests`, configure a request that overlaps with Request #1 (`Canon EOS 200D DSLR` has 2 total units, and 1 unit is already allocated for tomorrow):
   - **Requesting Club:** `3 — Drama Society`
   - **Resource to Borrow:** `#1 — Canon EOS 200D DSLR (Total: 2, 1 Allocated)`
   - **Quantity Needed:** `2`
   - **Start / End Window:** Set to tomorrow's date overlapping the active booking.
2. Click **"1. Check Time Conflict"**.
3. Point out the **Conflict Detected** box on the right:
   - MySQL procedure `CALL sp_check_scheduling_conflict` computes `Total Stock: 2 | Reserved in Window: 1 | Available: 1`.
   - Because `Available (1) < Needed (2)`, the API automatically executes `CALL sp_suggest_alternative_resources`, recommending available cameras in the same category (e.g., `Sony Alpha A6400 Mirrorless`).
4. Click **"Reserve Alternative"** to book the suggested alternative in one click.

Feature 5 • ACID Transactions + MySQL Triggers

5. Transactional Approval, Stock Validation Trigger & Auto-Notifications

1. Open `http://localhost:3000/coordinator` (**Coordinator & Maintenance**).
2. Locate a `pending` request and click **Approve**.
3. Explain the 3 database mechanisms that execute together:
   - **ACID Transaction:** Runs `BEGIN`, locks the request row with `SELECT ... FOR UPDATE`, updates `REQUEST.status = 'approved'`, inserts a new row into `ALLOCATION`, and runs `COMMIT`.
   - `BEFORE INSERT` **Trigger (** `trg_validate_allocation_stock` **):** Verifies that overlapping allocations do not exceed `quantity_total`. If exceeded, it raises `SIGNAL SQLSTATE '45000'` and rolls back the transaction.
   - `AFTER UPDATE` **Trigger (** `trg_notify_request_status` **):** Automatically inserts a notification into `NOTIFICATION` for every member of the requesting club (visible in the **Trigger-Generated Notifications** panel).
4. Click **Mark Returned** on an `allocated` row to mark it `returned` and restore available stock.

Feature 6 • MongoDB Validation + Cross-DB Sync

6. Equipment Maintenance Logging (MongoDB + MySQL Sync)

1. On `http://localhost:3000/coordinator`, use the **Log Equipment Maintenance** form:
   - **Resource ID:** `8` (`Yamaha Stagepas 400BT PA System`, currently `fair`)
   - **Issue Type:** `repair` | **Condition After:** `excellent` | **Cost Incurred:** `1200`
   - **Inspection / Repair Notes:** `Replaced mixer fader and calibrated stereo output.`
2. Click **Record Maintenance Log**. This inserts a schema-validated document into MongoDB's `maintenance_logs` collection and updates `RESOURCE.condition = 'excellent'` in MySQL.

Feature 7 • Complex SQL + MongoDB Aggregation Pipeline

7. Administrative Utilization & Analytics Dashboard

1. Open `http://localhost:3000/admin` (**Admin Analytics**).
2. Highlight the 4 analytical panels:
   - **Club Utilization Cards:** Queries MySQL view `vw_club_utilization_summary`.
   - **Frequently Requested Resources:** Multi-table `JOIN` with `GROUP BY`, `HAVING COUNT(req.request_id) >= 1`, and `ORDER BY`.
   - **Inter-Club Borrowing Patterns:** Aggregates exchanges between borrowing and lending clubs (`WHERE req.club_id <> r.club_id`).
   - **Underutilized Assets & MongoDB Maintenance Aggregation:** Displays `vw_underutilized_assets` alongside MongoDB's `$group` aggregation pipeline summarizing maintenance count and total repair cost per issue type.